

Due Sunday September 13th

1. In class we discussed a method for inverting matrices, that can be generalized to invert a non-square matrix. In the *singular value decomposition* (SVD) method, we use the fact that any $m \times n$ matrix $\mathbf{A}$ can be broken down as follows:

$$\mathbf{A} = \mathbf{U}\mathbf{S}\mathbf{V}^\dagger$$

where $\mathbf{U}$ is an $m \times m$ unitary matrix, $\mathbf{S}$ is an $m \times n$ positive diagonal matrix, and $\mathbf{V}$ is an $n \times n$ unitary matrix. Recall that a unitary matrix has the property that its inverse is its *conjugate transpose*, indicated with a dagger ($\dagger$). The inverse of $\mathbf{A}$ is given by

$$\mathbf{A}^{-1} = \mathbf{V}\mathbf{S}^{-1}\mathbf{U}^\dagger$$

Another method for inverting non-square matrices is the *QR* decomposition (QR) method. Here, we use the fact that any $m \times n$ matrix $\mathbf{A}$ can be broken down as follows:

$$\mathbf{A} = \mathbf{Q}\mathbf{R}$$

where $\mathbf{Q}$ is an $m \times m$ orthogonal matrix and $\mathbf{R}$ is an $m \times n$ upper triangular matrix. The inverse of $\mathbf{A}$ is given by

$$\mathbf{A}^{-1} = \mathbf{R}^{-1}\mathbf{Q}^{-1}$$

For this assignment, we are going to be doing a benchmark test of the two inversion methods. You should submit a Python script **benchmark.py** that does the following:

- For $i = 1, 10$
 - generate a random $2^i \times 2^i$ matrix
 - find inversion time t_{SVD} via the SVD method
 - find inversion time t_{QR} via the QR method
- Make a plot **benchmark.png** showing t_{SVD} and t_{QR} versus i .
- Print to standard output the time (in seconds) of inverting a 1024×1024 matrix by each method. Also print to standard output a statement about which method is the fastest.

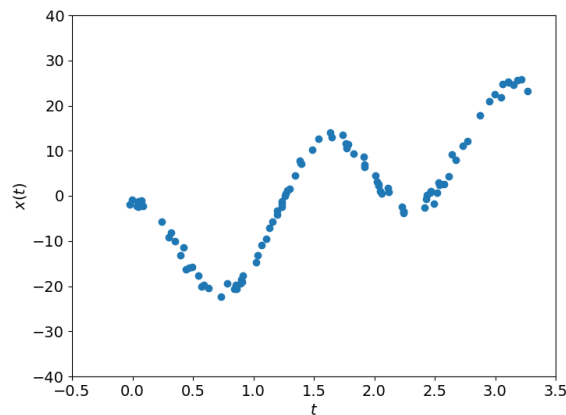
You should try to minimize the time taken by both methods. For example, $\mathbf{Q}$ is an orthogonal matrix; is it faster to explicitly invert it or take its transpose? Extra credit to anyone who can find a way of SV- or QR-inverting a random 1024×1024 matrix faster than the instructor.

2. In lab we discussed using the pseudoinverse technique to perform polynomial fits, but the method works in fitting data to other orthogonal sets of functions as well.

In the file **cosines.dat** are time-series (x vs t) data for a signal consisting of a sum of two cosine functions, namely:

$$x(t) = a_1 \cos t + a_4 \cos 4t$$

Note that the data is not sampled in equal timesteps, and contains random noise in x .



Create a Python script **cosines.py** to carry out the following:

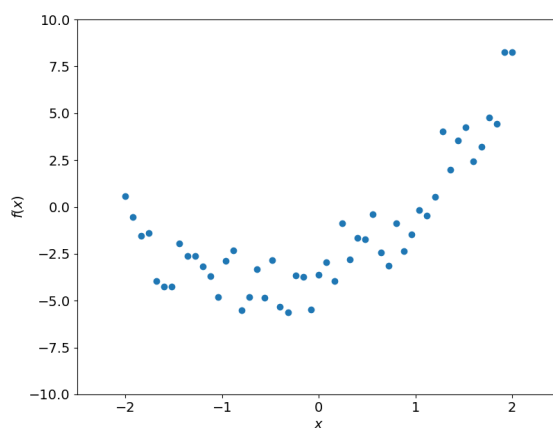
- Read in **cosines.dat**, assumed to be in the same directory as your Python script.
- Use the Vandermonde inversion technique (Lab 3) to estimate a_1 and a_4 .
- Your script should output a_1 and a_4 to standard output, and also plot your fit result overlaid on the data in a file called **cosines.png**.

Note: If you extend this approach and do some intelligent scanning over the possible frequencies, you can build up a frequency spectrum for data. This is only usually advantageous if the x-axis data is irregularly spaced. Such analysis is known as LSSA – least square spectrum analysis. For regularly spaced data, the FFT (fast Fourier transform) method is far superior.

3. The file **polyfile.dat** on the assignment page contains some $f(x)$ (second column) versus x (first column) data. f is a polynomial function of x , *i.e.*

$$f(x) = a_0 + a_1x + a_2x^2$$

with some normally-distributed noise on top of it. See the figure below. Your task is



to find the coefficients a_0 , a_1 and a_2 along with estimates of their errors.

You should submit a Python script **polyfit.py** that does the following:

- Read in **polyfile.dat**, assumed to be in the same directory as your Python script. Your code however should not be in any way specific to this particular file: I will change aspects (the coefficients, noise, N_{bins}) of the data when actually grading the assignment.
- Use the Vandermonde inversion technique (Lab 3) to estimate a_0 , a_1 and a_2 .
- Finally, since physical measurements are useless without uncertainties, you are to determine the uncertainty in a_0 , a_1 and a_2 using a bootstrap method. You will need to generate a number of simulated datasets which have the same a_0 , a_1 and a_2 and (randomized) noise as the original set. The uncertainties may then be taken to be the standard deviations of these quantities, taken from the simulated datasets.
- Your final output should be the fit quantities with your uncertainties, and plot written to **polyfit.png** showing your fit result overlaid on the data.

Submit your assignment by 11:59 PM on Sunday, September 13:

submit p4730 hw01 benchmark.py cosines.py polyfit.py